24CYS401

SECURE SOFTWARE ENGINEERING

INTEGRATED CAPSTONE PROJECT REPORT

SmartWaste Secure

IoT-Based Smart Waste Management and Secure Route Optimization

DOCUMENT CONTROL	VALUE
Student	Yeshwanth Raj S.
Register Number	CH.SC.U4CYS23052
Institution	Amrita Vishwa Vidyapeetham - Chennai Campus
Project	SmartWaste Secure / EcoSmart Autonomous Waste Management & IoT Security
Course	24CYS401 Secure Software Engineering
Semester	VII
Version	1.0
Date	08 October 2026
Document Type	Academic Capstone Engineering Report

Core Engineering Principle

The system does not trust IoT telemetry simply because a device is registered. Device identity is verified, telemetry is checked for integrity, freshness and physical plausibility, trusted data is used for route decisions, sensitive route operations are authorized and audited, and deployment is hardened through Docker, Kubernetes and CI/CD controls.

EXECUTIVE SUMMARY

Abstract and Project Context

SmartWaste Secure is an academic secure-software engineering system for IoT-based waste management. Smart bins periodically report fill level, location, temperature and sensor status. The platform receives and validates telemetry, detects suspicious behaviour, generates optimized collection routes and supports vehicle/driver assignment.

The uploaded project is implemented around a **FastAPI** backend, SQLAlchemy models, a static JavaScript frontend, an IoT simulator, Docker/Kubernetes deployment files and a GitHub Actions CI/CD workflow. The security implementation includes HMAC-SHA256 telemetry verification, replay checks, geofence/physical-plausibility checks, JWT authentication, role checks, IP rate limiting and audit logging.

> **Scope statement:** the report is an academic engineering model, not a certified municipal infrastructure deployment. Physical sensor models in the hardware section are a proposed laboratory prototype because the supplied problem statement specifies telemetry functions, not exact hardware part numbers.

DOCUMENT MAP

Table of Contents

Part I - Professional SRS

Part II - Phase 1 - Agile Process and Development Approach

Phase 2 - Requirements Engineering

Phase 3 - Requirements Analysis and UML

Phase 4 - Data and Information Flow Modeling

Phase 5 - Software Architecture and Design Engineering

Phase 6 - User Interface Design

Phase 7 - Threat Modeling and Security Analysis

Phase 8 - Attack Tree and Security Architecture Refinement

Phase 9 - Product Backlog and Jira / Scrum

Phase 10 - Sprint Execution and Scrum Metrics

Phase 11 - Secure Development and Build Environment

Phase 12 - Secure Coding and Refactoring

Phase 13 - Docker and Kubernetes

Phase 14 - CI/CD and Security Testing

Phase 15 - Logging, Monitoring, Hardening and Secure Deployment

Phase 16 - Final Security Review

Appendices - Traceability and Evidence

VISUAL MODELS INCLUDED

UML Use Case Diagram
Scenario / Sequence Model
ER Diagram
DFD Level 0
DFD Level 1
DFD Level 2 - Telemetry Processing
DFD Level 2 - Route Management
Trust Boundaries
Secure Architecture
Hardware / IoT Device Architecture
UI Model
Attack Tree
Kubernetes Security Model
CI/CD Pipeline

PART I - SOFTWARE REQUIREMENTS SPECIFICATION

1. SRS Overview

This SRS is the authoritative requirements baseline for the later phases. It defines the system scope, actors, interfaces, hardware proposal, functional requirements, quality attributes, security requirements, business rules and acceptance criteria.

1.1 Stakeholders and User Classes

ACTOR	ROLE	ACCESS / SECURITY RESPONSIBILITY
Administrator	Manage devices, bins, vehicles, drivers and routes.	Strong authentication; privileged operations audited.
Operations Operator	Monitor bins and generate/dispatch routes.	Operational permissions; no unnecessary security privilege.
Driver	View assigned route and update collection status.	Only assigned operational data; no route-admin privilege.
Security Analyst	Review security events and use security-lab functions.	Security/audit access; not routine fleet administration.
Smart Bin	IoT device providing telemetry.	Device identity and telemetry-only privilege.
Faculty / Evaluator	Reviews engineering evidence.	Assessment stakeholder; not an application actor.

1.2 System Scope

In scope: device registration/lifecycle, telemetry ingestion, anti-spoofing, bin monitoring, route optimization, vehicle/driver assignment, RBAC, audit logging, UI, Docker/Kubernetes and CI/CD security.

Out of scope: industrial hardware certification, autonomous vehicle control, production municipal SLA, government identity integration and national-scale infrastructure.

1.3 Hardware / IoT Device Baseline

The problem statement requires four periodic data categories. The following is the proposed lab device stack:

COMPONENT	ROLE	REQUIREMENT / NOTE
ESP32-WROOM	Edge MCU + Wi-Fi	Reads sensors, builds telemetry and connects to the backend.
HC-SR04 ultrasonic	Fill-level sensor	Distance measurement converted to fill percentage.
NEO-6M GPS	Location sensor	Latitude/longitude for bin position.
DS18B20	Temperature sensor	Temperature reading in degrees C.
ATECC608A secure element	Target device-key protection	Proposed hardware-backed identity; not present in current simulator implementation.
Protected enclosure	Physical protection	Restrict access to controller and provisioning/debug interface.
Mosquitto MQTT broker	IoT message transport	Proposed secure broker path; current simulator posts directly to HTTP endpoint.

1.4 Hardware Interface Requirements

ID	INTERFACE	DATA	REQUIREMENT
HI-01	HC-SR04 -> ESP32 / GPIO	Distance	Controller shall sample distance and convert it to fill-level input.
HI-02	NEO-6M -> ESP32 / UART	Coordinates	Controller shall read location when available.
HI-03	DS18B20 -> ESP32 / 1-Wire	Temperature	Controller shall read and report temperature.

ID	INTERFACE	DATA	REQUIREMENT
HI-04	ATECC608A -> ESP32 / I2C	Cryptographic operations	Target prototype should use secure-element key operations.
HI-05	ESP32 -> MQTT/HTTPS	Telemetry	Device shall transmit authenticated telemetry over a protected channel.

1.5 Functional Requirements

ID	REQUIREMENT	PRIORITY
FR-01	Authenticate users before protected functions.	Critical
FR-02	Register, activate and revoke smart bins.	High
FR-03	Authenticate smart-bin telemetry with device integrity checking.	Critical
FR-04	Receive fill level, location, temperature and sensor status.	Critical
FR-05	Validate telemetry schema, types and physical bounds.	Critical
FR-06	Store accepted telemetry with device association and timestamp.	High
FR-07	Display bin status and alerts.	High
FR-08	Detect suspicious sensor/location behaviour.	Critical
FR-09	Generate optimized collection route from validated bins.	Critical
FR-10	Assign vehicles and drivers to routes.	High
FR-11	Allow driver to view assigned route and update collection status.	High
FR-12	Restrict route modification/cancellation to authorized users.	Critical
FR-13	Record security-sensitive and administrative actions.	High
FR-14	Generate security alerts for spoofing/anomalies/unauthorized activity.	High
FR-15	Apply rate limiting to protect service availability.	High
FR-16	Expose device last-seen/health state.	Medium

1.6 Non-Functional Requirements

ID	CATEGORY	REQUIREMENT
NFR-01	Security	Protected communication shall use TLS where deployed.
NFR-02	Authorization	Sensitive operations shall be enforced server-side and deny by default.
NFR-03	Integrity	Invalid telemetry shall not alter trusted operational state.
NFR-04	Availability	Rate limiting, health checks and bounded resources shall reduce abuse impact.
NFR-05	Auditability	Sensitive operations shall identify actor/device, resource, time and outcome.
NFR-06	Maintainability	Authentication, anti-spoofing, routing and persistence shall remain modular.
NFR-07	Portability	The system shall be reproducible in a local containerized laboratory environment.
NFR-08	Usability	Roles shall see only relevant functions and receive clear feedback.

1.7 Security Requirements

ID	SECURITY REQUIREMENT	PROPERTY
SEC-01	Unique device identity and lifecycle state.	Authentication / Integrity

ID	SECURITY REQUIREMENT	PROPERTY
SEC-02	HMAC-SHA256 signature verification for telemetry.	Integrity / Authentication
SEC-03	Replay/freshness protection for telemetry.	Integrity
SEC-04	Fill/temperature/location validation and plausibility checks.	Integrity
SEC-05	Geofence/physical movement anomaly detection.	Integrity
SEC-06	JWT-based user authentication.	Authentication
SEC-07	Role-based authorization for administration/dispatch.	Authorization
SEC-08	Object-level authorization for route resources.	Authorization
SEC-09	IP-based rate limiting / abuse protection.	Availability
SEC-10	No secrets/tokens/passwords in logs.	Confidentiality
SEC-11	Audit route/device/security-sensitive actions.	Accountability
SEC-12	Least-privilege database access.	Confidentiality / Integrity
SEC-13	Non-root and restricted container runtime.	Defense in depth
SEC-14	Kubernetes resource and network restrictions.	Availability / Least privilege

1.8 CIA and Authentication / Authorization / Audit

AREA	SMARTWASTE REQUIREMENT
Confidentiality	Protect credentials, device keys, route information, driver data, security logs and configuration.
Integrity	Protect telemetry, coordinates and route state from unauthorized or implausible manipulation.
Availability	Protect telemetry and API endpoints from request flooding and resource exhaustion.
Authentication	User JWT/bcrypt; device HMAC-SHA256 telemetry verification.
Authorization	Backend role checks; route modification restricted to privileged roles; driver is isolated in UI.
Audit	Login failures/success, spoofing, location anomaly, route creation and assignment, device lifecycle and rate-limit events are defined in AuditAction.

1.9 Data Entities

ENTITY	PURPOSE
User	Account identity, password hash, role and active state.
SmartBin	Device identity, key, nominal location, capacity, zone and status.
SensorReading	Fill, location, temperature, sensor status, battery and anomaly data.
Vehicle	Vehicle identity, plate, type, capacity, fuel and availability.
Driver	Driver identity, license, contact and availability.
CollectionRoute	Optimized route, ordered bin sequence, distance, estimated time, status and version.
RouteAssignment	Associates route, vehicle and driver with schedule/status.
AuditLog	Security/operational event evidence.
DeviceToken	Device-token lifecycle metadata.

1.10 Acceptance Criteria

ID	ACCEPTANCE CRITERION
AC-01	Valid user authenticates; invalid credentials are rejected without sensitive disclosure.
AC-02	Valid HMAC telemetry is accepted; invalid signature is rejected and audited.
AC-03	Replay/stale telemetry is rejected by the replay check.
AC-04	Location outside the 50m registered geofence is rejected and audited.
AC-05	Unauthorized route change is rejected at the API authorization layer.
AC-06	Excessive requests trigger HTTP 429 rate limiting.
AC-07	Critical events are persisted in AuditLog.
AC-08	Container/Kubernetes deployment runs without root privileges.

> **SRS conclusion:** this requirements baseline deliberately separates *what the system must do* from *how the laboratory prototype currently implements it*. The hardware part numbers are therefore identified as a proposed physical realization, while the uploaded repository remains simulator/software based.

PHASE 1

6 Marks

Agile Process and Development Approach

> **Required activity:** select Agile approach, map at least five Manifesto principles, identify two refactoring opportunities, and state two limitations/risks with mitigations.

1.1 Selected approach and justification

Hybrid Scrum + XP is selected. Scrum supplies planning, sprint cadence, review and retrospective discipline; XP contributes test-first thinking, continuous integration, peer review and refactoring for security-sensitive modules. This matches the supplied project document and the uploaded repository's GitHub/Jira workflow.

1.2 Manifesto mapping

PRINCIPLE	CONCRETE SMARTWASTE APPLICATION
Customer collaboration	Operations and security stakeholders validate route dispatch, driver manifest and security-lab behaviour.
Working software	Progress is represented by executable FastAPI, frontend, route engine, simulator, tests and deployment files.
Frequent delivery	Jira defines timeboxed sprint increments for anti-spoofing, route engine, driver manifest and RBAC/gateway.
Technical excellence	HMAC, replay checks, geofence validation, RBAC, rate limiting and CI scanning are backlog items.
Simplicity	Greedy Nearest-Neighbor routing is used instead of a heavier optimization solver for the lab.
Responding to change	Threat findings and security gaps become new backlog/refactoring work.

1.3 Refactoring opportunity A - route optimizer

Before: `optimize_route()` combines database queries, geodesic calculations, route algorithm and persistence in one function.

After: extract a pure `RoutingStrategy`/ `NearestNeighborTSPOptimizer` and keep DB persistence in a separate service.

1.4 Refactoring opportunity B - anti-spoofing pipeline

Before: `process_telemetry_security()` contains sequential HMAC, replay and location checks.

After: Chain-of-Responsibility style security steps so checks are independently testable and extendable.

1.5 Agile risks and mitigations

RISK	MITIGATION
Speed can cause security shortcuts.	Security Definition of Done: critical security tests, SAST and image checks must pass before release.
Hardware constraints can be ignored by software sprint changes.	Maintain a simulator/digital twin and an architectural runway; validate cryptographic and edge assumptions before physical rollout.

Source evidence

docs/ agile_process_and_development_approach.md

Jira evidence

Four epics and three named sprints are documented.

Important distinction

The repository's live implementation is the evidence baseline; future architectural refactors are recommendations unless committed.

PHASE 2

7 Marks

Requirements Engineering

The standalone SRS in Part I is the complete Phase 2 deliverable. The following short summary shows how it feeds later phases.

Stakeholders

Administrator, Operator, Driver, Security Analyst, Smart Bin, Faculty/Evaluator.

Requirement classes

16 functional, 8 NFR and 14 security requirements.

Security baseline

CIA + authentication + authorization + audit + device security are explicit.

PHASE 3

7 Marks

Requirements Analysis and UML

3.1 UML Use Case Diagram

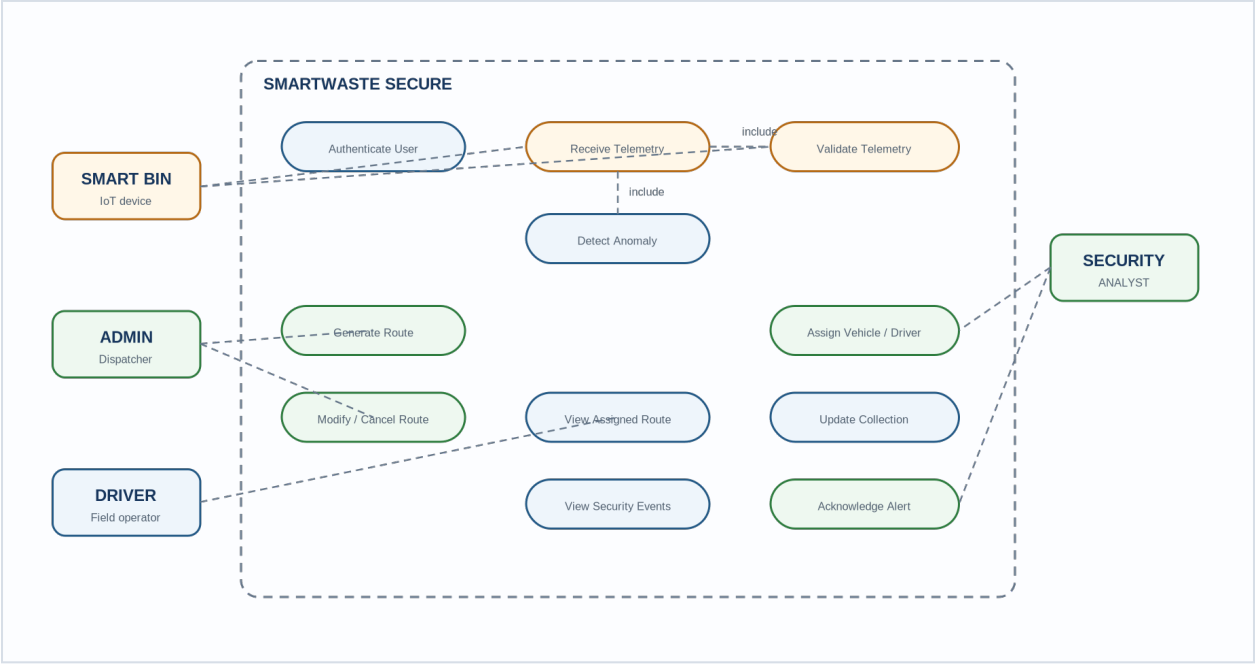


Figure 1 - SmartWaste Secure Use Case Diagram.

3.2 Critical Use Case - Submit Telemetry

FIELD	SPECIFICATION
Actor	Smart Bin
Preconditions	Registered/active device; valid connectivity.
Main flow	Send telemetry -> locate bin record -> verify HMAC -> replay check -> geofence/physical check -> data sanity -> persist reading.
Alternative	Suspicious data can be rejected/flagged and audited.
Exception	Unknown bin, invalid signature, replay, geofence failure or invalid physical values.
Postcondition	Trusted reading is persisted; security event is created for security failures.

3.3 Critical Use Case - Generate / Assign Route

FIELD	SPECIFICATION
Actor	Administrator / Operator
Preconditions	Authorized user; eligible bins available.
Main flow	Authenticate -> retrieve bins above collection threshold -> optional zone filter -> run geodesic nearest-neighbor -> create route -> assign vehicle/driver -> audit.
Alternative	Generate route for a selected zone.
Exception	No eligible bins; missing resource; insufficient role permission.
Postcondition	Route and assignment are persisted and driver receives route access.

3.4 Scenario-based analysis

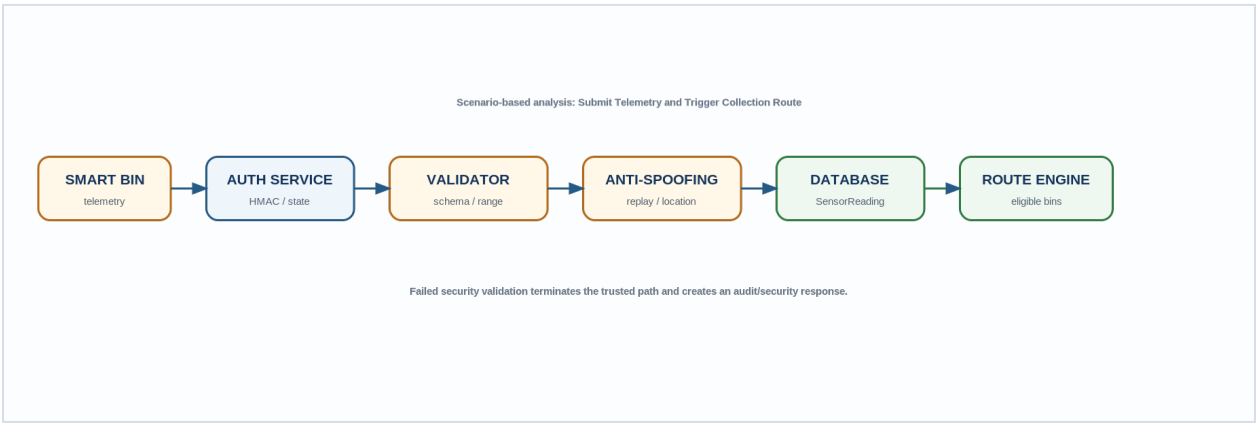


Figure 2A - Scenario-based analysis: Submit Telemetry and Trigger Collection Route.

The examination template names “Attend Examination and Submit Answers.” Since the entire exercise must be derived from the SmartWaste case, the consistent domain scenario is **“Submit Telemetry and Trigger Collection Route.”**

> **Scenario:** Smart Bin -> authenticated telemetry -> validation -> route eligibility -> route optimization -> administrator review -> vehicle/driver assignment -> audit.

PHASE 4

7 Marks

Data and Information Flow Modeling

4.1 ER Diagram

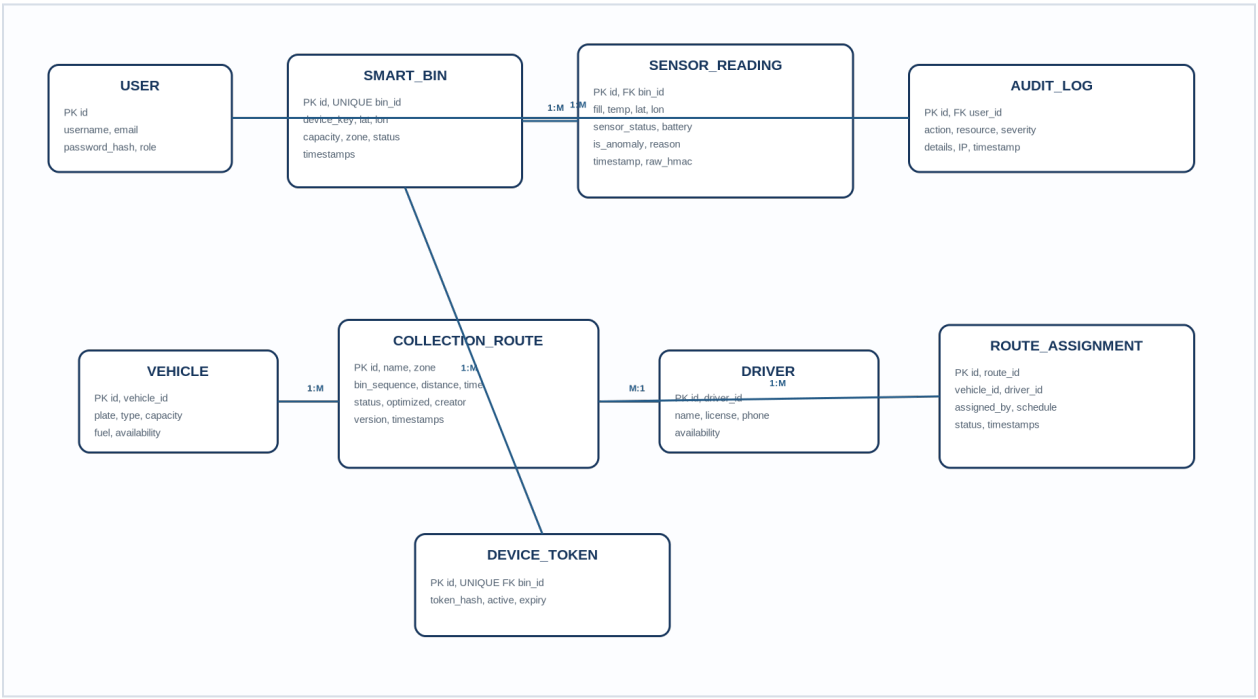


Figure 2 - Logical ER model derived from the uploaded SQLAlchemy models.

4.2 Level-0 DFD



Figure 3 - DFD Level 0 - context.

4.3 Level-1 DFD

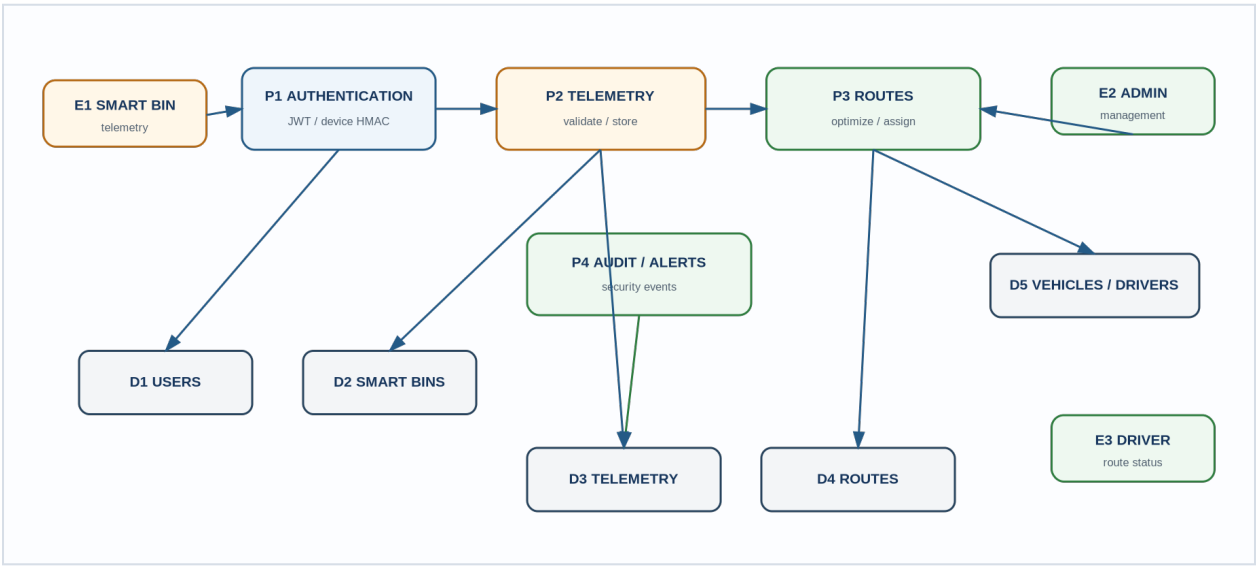


Figure 4 - DFD Level 1 - processes and data stores.

4.4 Trust boundaries

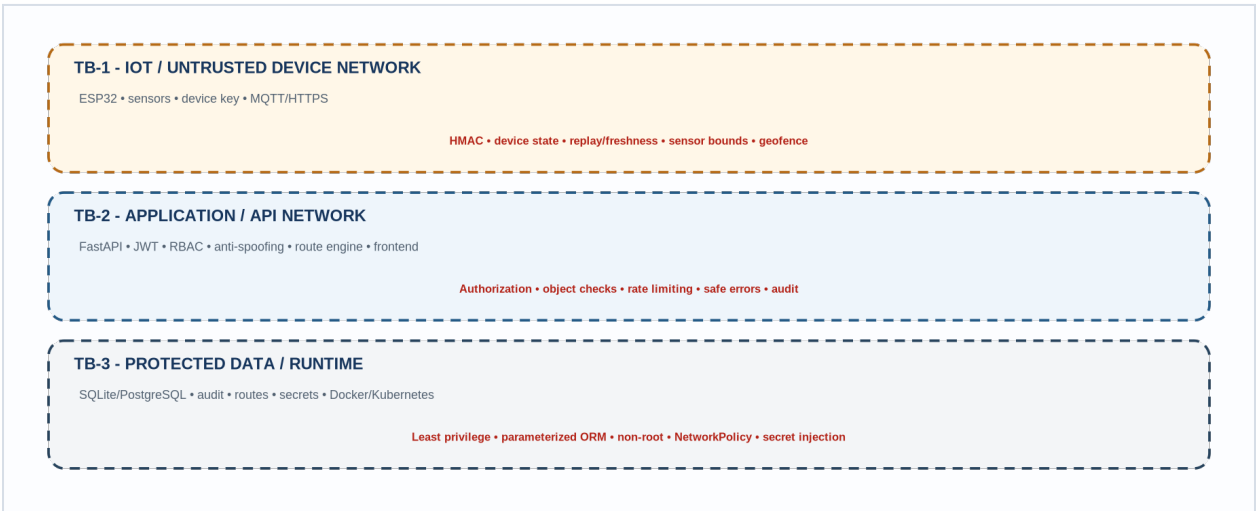


Figure 5 - SmartWaste trust boundaries.

4.5 Consistency statement

Use cases map to data and process elements: Smart Bin -> SmartBin/SensorReading; route generation -> CollectionRoute; dispatch -> RouteAssignment; security evidence -> AuditLog; identity -> User/DeviceToken. The DFD processes and stores use the same vocabulary.

PHASE 5

7 Marks

Software Architecture and Design Engineering

5.1 Architecture style and justification

The uploaded repository is a **layered FastAPI application** rather than Flask. The practical layers are presentation/frontend, FastAPI API boundary, authentication/authorization, anti-spoofing/routing domain services, SQLAlchemy persistence and runtime/container infrastructure.

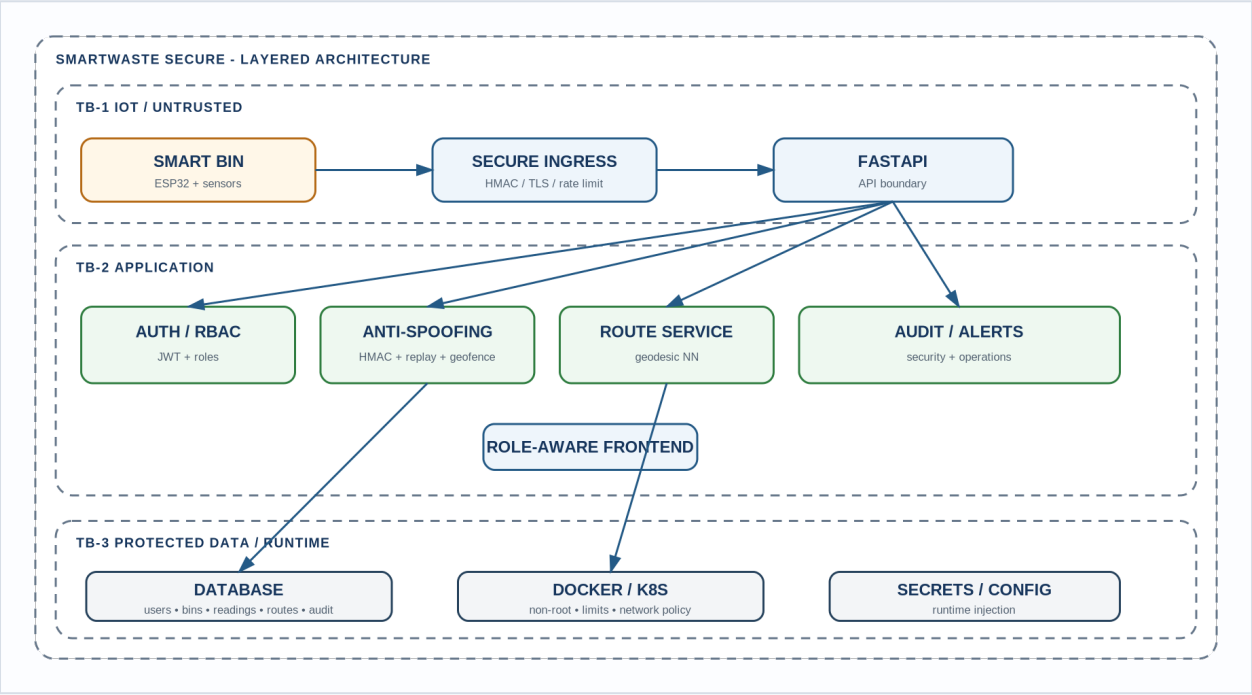


Figure 6 - Secure architecture based on the uploaded implementation.

5.5 Hardware and IoT Device Architecture

The physical sensor stack is a proposed laboratory prototype; the current uploaded repository uses the IoT simulator.

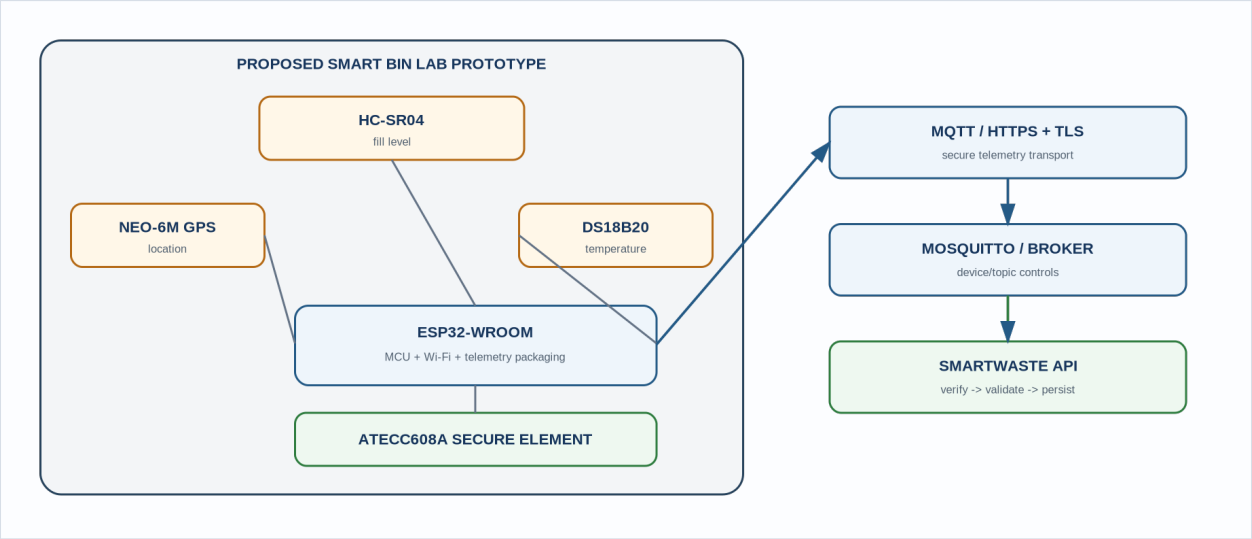


Figure 9A - Proposed physical smart-bin architecture.

5.2 Major components and interfaces

COMPONENT	RESPONSIBILITY	INTERFACE
FastAPI main.py	HTTP endpoints, CORS middleware, rate limit middleware, frontend mount.	REST API
auth.py	bcrypt password hashing, JWT creation/verification, role checks, audit helper.	JWT/Bearer dependency
schemas.py	Pydantic validation for users, bins, readings, routes and assignments.	Typed API models
anti_spoofing.py	HMAC verification, replay cache, geofence and physical plausibility.	Telemetry security pipeline
route_optimizer.py	Threshold selection + geodesic Nearest-Neighbor route generation.	Route service
rate_limiter.py	In-memory IP sliding-window rate limiting.	HTTP middleware
models.py	SQLAlchemy data entities.	Database ORM
Frontend	Dashboard, role navigation, map, TSP controls, driver manifest, security lab.	HTTP/API + static assets

5.3 Design patterns / concepts

PATTERN / CONCEPT	APPLICATION
Layered Architecture	Separates API, security, domain and persistence responsibilities.
Dependency Injection	FastAPI dependencies provide DB sessions and current-user checks.
Strategy Pattern	Recommended separation of routing algorithm from route persistence.
Chain of Responsibility	Recommended decomposition for telemetry security checks.
Middleware	Rate limiting and security headers are implemented at HTTP middleware layer.

5.4 Domain-to-component mapping

CONCERN	COMPONENT
Authentication	auth.py + JWT dependency
Telemetry verification	anti_spoofing.py
Operational evaluation	route_optimizer.py + telemetry thresholds
Route management	main.py route endpoints + CollectionRoute / RouteAssignment

CONCERN	COMPONENT
Audit	AuditLog + log_audit()

PHASE 6

5 Marks

User Interface Design

The uploaded frontend implements role-aware navigation and views for administrator, operator, driver and analyst profiles. The UI includes dashboard, map, routes, security logs and Security Gateway Lab.

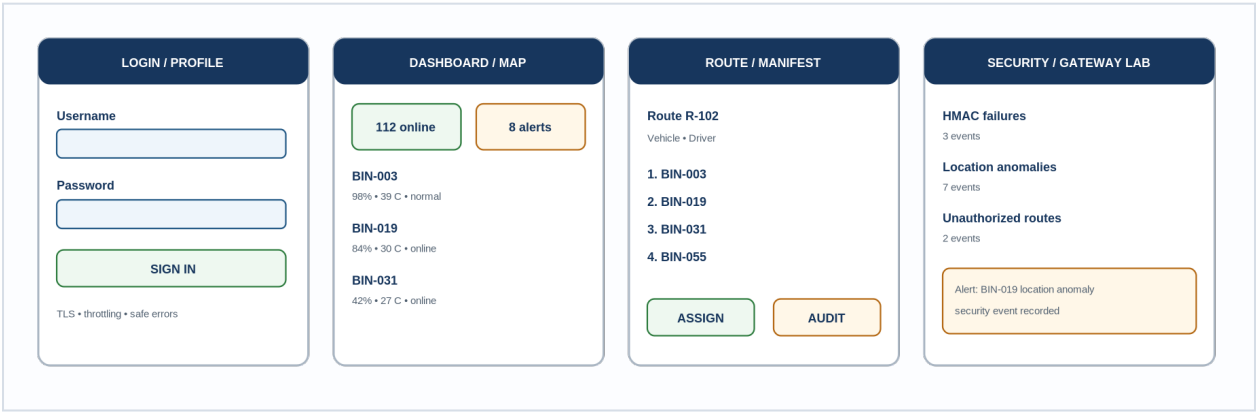


Figure 7 - UI architecture derived from frontend/index.html and frontend/app.js.

SCREEN	USER / GOAL	SECURITY / USABILITY
Dashboard	Admin/Operator - monitor operations.	Visibility of KPIs; role-aware navigation.
Map / Bin discovery	Admin/Operator - identify bins and vicinity.	Operational data filtered by role.
Routes / Manifest	Admin/Operator/Driver - execute collection.	Driver partitioned from security sections.
Security Gateway Lab	Analyst - demonstrate HMAC/geofence controls.	Security-only navigation and terminal evidence.
Access Control Barrier	Any unauthorized profile.	Clear denial feedback without exposing internals.

PHASE 7

10 Marks

Threat Modeling and Security Analysis

7.1 Assets / CIA

ASSET	C	I	A	RISK
Device key / HMAC material	High	Critical	High	Device spoofing
Sensor telemetry	Medium	High	High	False operational state
Bin location	High	Critical	High	Route corruption
Collection route	High	Critical	High	Physical dispatch impact
User credentials	High	High	Medium	Account takeover

ASSET	C	I	A	RISK
Vehicle/driver assignment	Medium	High	High	Dispatch error
Audit logs	High	Critical	High	Loss of accountability
Secrets / config	Critical	Critical	High	Platform compromise
Database	High	Critical	Critical	Central data loss
Security alerts	High	High	High	Delayed response

7.2 STRIDE analysis

ELEMENT	THREAT	STRIDE	MITIGATION
Smart Bin -> API	Forged device identity	Spoofing	HMAC/device identity + revocation.
Telemetry payload	Changed fill/temp values	Tampering	Pydantic + anti-spoofing validation.
Location	Fake / moved coordinates	Tampering	50m geofence + physical plausibility.
Telemetry timestamp	Replay valid data	Spoofing/Tampering	5-minute replay window.
Route API	Unauthorized state change	Elevation/Tampering	Role checks + object-level authorization.
Security log	Action denial / loss of trace	Repudiation	AuditLog.
API/middleware	Request flood	Denial of Service	IP rate limiting.
Secrets	Key exposure	Information Disclosure	Runtime secret management.
Container	Root compromise	Elevation of Privilege	Non-root + dropped capabilities.
Kubernetes service	Unwanted network access	Information Disclosure / DoS	ClusterIP + NetworkPolicy.

7.3 Sensitive information flows

FLOW	SECURITY CHECKPOINTS
Device key -> HMAC -> telemetry endpoint	Signature verification; device state; replay window.
GPS -> Pydantic -> geofence/physical check -> route selection	Bounds; 50m registered anchor; >100 km/h jump detection.
Admin -> route API -> route/assignment store -> driver	JWT; role authorization; audit.

7.4 Vulnerabilities found in uploaded repository

ID	OBSERVED GAP	SEVERITY	REQUIRED FIX
V-01	GET /api/bins, /api/routes and /api/security/logs are not protected by user authentication in main.py.	High	Apply get_current_user/permission dependency to protected reads.
V-02	CORS uses wildcard origin with credentials enabled.	High	Restrict origins to trusted frontend hosts.
V-03	SmartBin stores device_key directly and simulator contains a literal device key.	Critical	Use secure provisioning / hardware-backed key storage; remove committed demo secrets.
V-04	Telemetry replay timestamp is derived from server time rather than a device-signed timestamp.	High	Include device timestamp/sequence in signed payload.
V-05	Replay cache is process-local memory and not shared across replicas.	Medium	Use shared cache/state for multi-replica deployment.
V-06	Kubernetes Secret contains static base64 values in manifest.	Critical	Inject secrets at deployment/runtime.

ID	OBSERVED GAP	SEVERITY	REQUIRED FIX
V-07	docker-compose.yml contains example SECRET_KEY/HMAC_SECRET values.	High	Move to external secret store and prevent commits.
V-08	Trivy exit-code is 0, so scan findings do not block build.	High	Fail release on prohibited severity thresholds.
V-09	Current route code exposes creation/assignment, but a dedicated route-modification endpoint with object-level authorization is not implemented.	High	Implement authenticated route update/cancel with resource authorization and audit.
V-10	Docker healthcheck calls curl but the slim image does not explicitly install curl.	Medium	Use a Python/HTTP healthcheck or install curl intentionally.

Key review finding: the project has a strong security concept and several implemented controls, but the report should not claim the system is “100% secure.” The repository contains real gaps that are ideal for Phase 7 -> Phase 8 -> remediation -> retest traceability.

PHASE 8

6 Marks

Attack Tree and Security Architecture Refinement

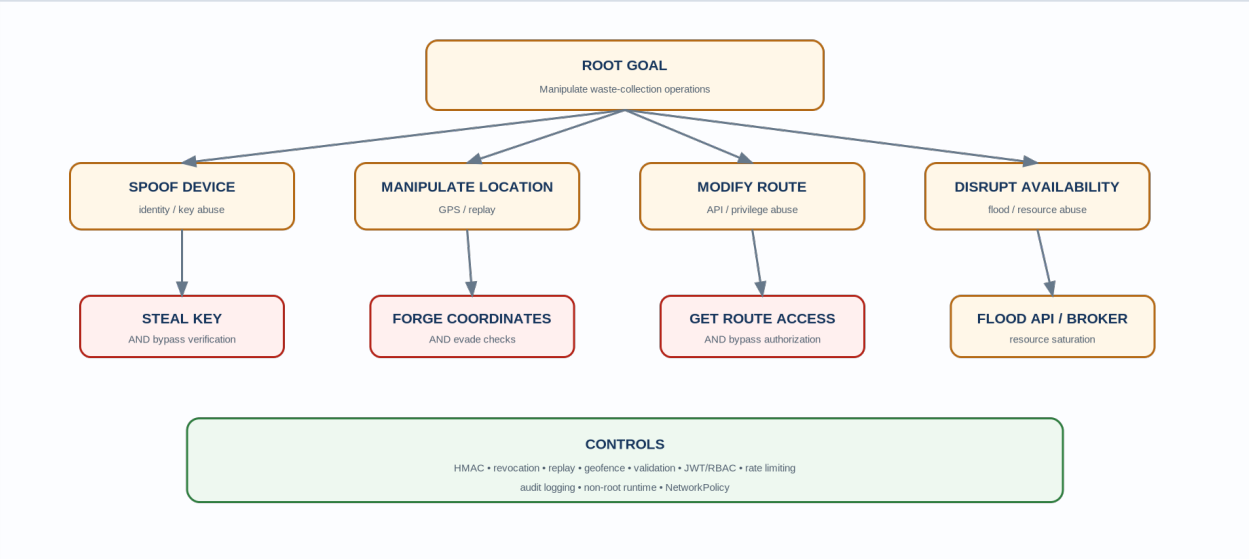


Figure 8 - Attack Tree for operational manipulation.

ATTACK PATH	REFINEMENT
Spoof device	HMAC verification + revocation + protected device identity.
Manipulate location	Geofence + physical plausibility + signed timestamp/sequence.
Modify route	JWT + role permission + object-level authorization + versioning + audit.
Disrupt availability	Rate limiting + broker/topic restrictions + resource limits + health monitoring.

Security architecture rule: the high-impact path must be ordered as **authenticate -> authorize -> validate -> persist -> audit**. For device telemetry: **device identity -> HMAC -> freshness/replay -> location -> sanity -> persist -> alert.**

PHASE 9

7 Marks

Product Backlog and Jira / Scrum

9.1 Product backlog - 12 stories

ID	EPIC	STORY SUMMARY	PRIORITY	POINTS
US-01	Sensor Security	HMAC-SHA256 telemetry verification	Highest	8
US-02	Sensor Security	50m geofence anomaly tripwire	High	5
US-03	Telemetry	Schema and physical-bounds validation	High	5
US-04	Device Lifecycle	Device revoke/reactivate flow	High	3
US-05	Route Engine	Geodesic Nearest-Neighbor TSP optimizer	High	8
US-06	Driver Manifest	Waypoint checklist and collection progress	High	5
US-07	RBAC	Driver vs security analyst partition	Highest	5
US-08	Security Gateway	HMAC/geofence attack laboratory UI	Medium	5
US-09	Route Security	Object-level route update authorization	Highest	5
US-10	Audit	Security-sensitive action logging	High	3
US-11	Availability	Rate limiting and resource controls	High	3
US-12	DevSecOps	Automated SAST/test/container security gates	High	5

9.2 Actual Jira baseline from project files

EPIC	NAMED JIRA / IMPLEMENTATION SCOPE
ECO-3 Sensor Security	HMAC-SHA256 anti-spoofing + geofence.
ECO-4 Route Engine	Nearest-Neighbor TSP + driver manifest.
ECO-5 RBAC Isolation	Driver vs security analyst partition + security gateway lab.
ECO-6 DevSecOps CI/CD	SAST + container scanning + Kubernetes rollout.

9.3 Sprints in supplied Jira artifacts

SPRINT	GOAL	POINTS / STATUS
ECO Sprint 1: Security & Auth	HMAC, geofence, CI/CD gate.	16 points / closed
ECO Sprint 2: Route & Fleet	TSP route + driver manifest.	13 points / active
ECO Sprint 3: RBAC & Gateway	Profile isolation + security gateway.	10 points / future

> **Source-of-truth note:** Jira artifacts report **39 story points** across the three named sprints. For the exam's two-sprint planning requirement, group Sprint 1 as the first assessment increment and Sprint 2 as the second; Sprint 3 remains a follow-on implementation sprint.

PHASE 10

7 Marks

Sprint Execution and Scrum Metrics

10.1 Board workflow

TO DO
Backlog-ready story.

IN PROGRESS
Development underway.

TESTING
Functional/security verification.

10.2 Required Scrum records

Each sprint should retain Daily Scrum notes, board state, burndown, velocity, defect list, Sprint Review and Retrospective. The supplied Jira files provide sprint names/goals and story-point scope; they do not contain a reliable daily burndown dataset in the repository, so no invented burn-down curve is asserted here.

10.3 Sprint review and retrospective focus

SPRINT	REVIEW FOCUS	RETROSPECTIVE ACTION
Sprint 1	HMAC + geofence + CI gate.	Sign device timestamp and move secrets out of code/config.
Sprint 2	TSP route + driver manifest.	Separate route domain logic from DB persistence; close route authorization gap.
Sprint 3	RBAC + Security Gateway Lab.	Apply RBAC to protected GET endpoints and make security scans release-blocking.

10.4 Metrics

{tbl(["Metric","Current source-backed information"],[("Total delivered scope","39 story points are recorded in the Jira implementation summary."), ("Velocity","Actual sprint velocity should be taken from Jira completed points; the three-sprint point totals are scope evidence, not daily velocity measurements."), ("Defects","Use actual Jira/test defect records; do not invent counts."), ("Carry-over","Use actual sprint completion state from Jira.")])}

PHASE 11

6 Marks

Secure Development and Build Environment

11.1 Repository/workflow

Git/GitHub is used with `main` as the primary branch. GitHub Actions is triggered on push/pull request. A secure working model is feature branch -> pull request -> security checks -> review -> merge.

11.2 Secure-build controls

CONTROL	PROJECT STATUS
Least privilege	Backend role checks exist; database access is application mediated. Protected GET routes still need authentication.
Secret management	Environment-based configuration exists, but committed example secrets remain in simulator/compose/Kubernetes.
Dependency control	backend/requirements.txt pins package versions.
SAST	Bandit is configured in GitHub Actions.
Container scanning	Trivy is configured.

CONTROL	PROJECT STATUS
Branch protection	Recommended in GitHub; evidence should be screenshot/captured from settings.
Reproducible build	Dockerfile + pinned dependencies provide a repeatable build path.

11.3 Security-check remediation

CHECK	CONFIGURED COMMAND/ ACTION	ACTION REQUIRED
Bandit	bandit -r backend/app -ll	Keep as mandatory gate.
pytest	pytest -v	Keep as merge gate; local analysis run was blocked by missing python-jose package even though requirements.txt declares it.
Fuzzing	backend/tests/fuzz_test.py	Keep as pre-release security regression.
Trivy	CRITICAL,HIGH, but exit-code 0	Change to blocking policy for release.

PHASE 12

4 Marks

Secure Coding and Refactoring

12.1 Secure module selected

The selected module is **IoT telemetry ingestion**, because it is the direct security boundary between smart-bin input and trusted application state. The repository already contains a before/after refactoring pair in `backend/refactoring/`.

12.2 Initial security weaknesses

WEAKNESS	IMPACT
Raw SQL / string-built query	Creates SQL-injection risk in the vulnerable example.
No authentication/integrity check	Allows unauthorized telemetry injection.
No physical validation	Impossible values can enter the decision path.
Weak exception handling	May expose internals or fail unclearly.

12.3 Refactored design

> **Before:** raw request values, direct SQL/string construction, no cryptographic device verification and weak validation.

> **After:** HMAC-SHA256 verification, Pydantic validation, ORM/parameterized persistence, controlled exceptions and audit logging.

12.4 Security code pattern

```
@app.post("/api/iot/telemetry")
def receive_telemetry(reading: SensorReadingCreate, ...):
    # Input has already passed typed Pydantic validation.
    # The security pipeline then verifies integrity and freshness.
    is_valid, error_reason = process_telemetry_security(
        db, bin_record, reading.dict(),
        reading.hmac_signature, timestamp_unix, client_ip
    )
```

```
    if not is_valid:
        raise HTTPException(
            status_code=400,
            detail=f"Telemetry rejected: {{error_reason}}"
        )
    # Persist only after the security pipeline succeeds.
```

The refactoring target is not cosmetic: it reduces injection and invalid-data risk while making security checks easier to test and extend.

PHASE 13

7 Marks

Containerized Development - Docker and Kubernetes

13.1 Docker implementation

The repository uses a two-stage `python:3.12-slim` Docker build, creates a non-root user `appuser` with UID/GID 10001, copies the virtual environment into the runtime image and starts Uvicorn on port 8000.

CONTROL	REPOSITORY EVIDENCE
Minimal base image	python:3.12-slim.
Non-root execution	USER appuser.
Read-only root filesystem	docker-compose.yml uses read_only: true.
No-new-privileges	docker-compose.yml uses no-new-privileges.
Resource limits	compose sets 512 MB memory and 1 CPU.
Ephemeral temp storage	compose uses tmpfs for /tmp.

13.2 Kubernetes configuration

CONTROL	REPOSITORY EVIDENCE
Namespace	smart-waste.
Scale	Deployment replicas: 3.
Non-root	runAsNonRoot=true; UID/GID 10001.
Privilege reduction	allowPrivilegeEscalation=false; capabilities drop ALL.
Filesystem	readOnlyRootFilesystem=true.
Resources	CPU and memory requests/limits.
Network	ClusterIP Service + NetworkPolicy.
Secrets	Kubernetes Secret object exists; values are currently static base64 and need deploy-time replacement.

13.3 Deployment architecture

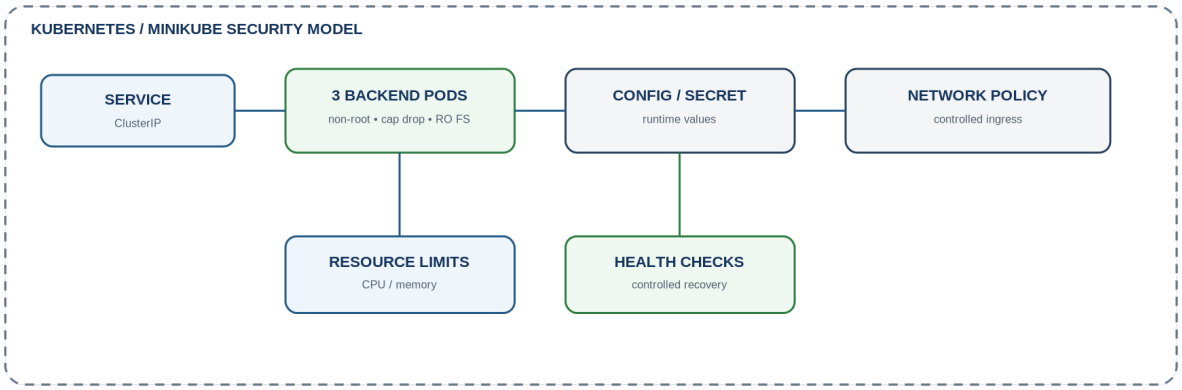


Figure 6 - Secure application/runtime architecture used for Docker/Kubernetes mapping.

> **Hardening gap:** the repository's static Kubernetes secret values and local compose example secrets should be replaced by deployment-time secret injection before a final production-style deployment claim.

PHASE 14

CI/CD and Security Testing

7 Marks

14.1 CI/CD pipeline

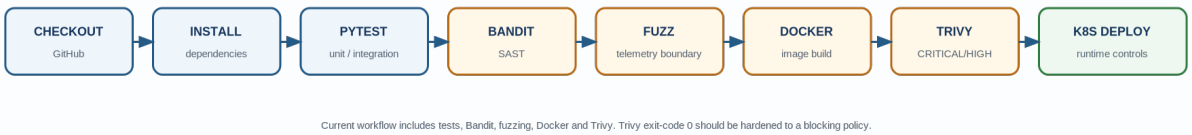


Figure 7 - CI/CD pipeline represented by .github/workflows/ci-cd.yml.

The current workflow performs dependency installation, Bandit SAST, pytest, fuzz testing, Docker image build and Trivy image scanning. The container build waits for the test-and-scan job.

14.2 Unit tests

TEST	WHAT IT CHECKS
test_password_hashing	bcrypt hash/verification correctness.
test_device_signature_verification	Correct HMAC passes; invalid signature fails.
test_replay_attack_prevention	Repeated/expired timestamp is detected.
test_api_rate_limiting	Repeated calls eventually receive 429.

14.3 Integration / fuzz testing

TEST	EXPECTED BEHAVIOUR
Valid telemetry integration	HTTP 201 and reading accepted.
Invalid signature integration	HTTP 400 and signature rejection.
Type mismatch fuzz	Validation rejection, normally HTTP 422.
Oversized ID fuzz	Validation rejection.
SQLi-shaped ID fuzz	Safe rejection; no SQL execution.

14.4 Current verification status

> **Execution note:** a local pytest run in the current analysis environment could not collect the test modules because `python-jose` was unavailable. The repository's `requirements.txt` declares `python-jose[cryptography]==3.3.0`, and external package installation was unavailable in this environment. Therefore no false “all tests passed” claim is made.

14.5 Defect -> fix -> retest

DEFECT	SEVERITY	FIX	RETEST
Server time is used for telemetry freshness rather than a device-signed timestamp.	High	Sign device timestamp/sequence and validate at gateway.	Replay/stale timestamp test must reject.
Trivy uses exit-code 0.	High	Use a non-zero policy threshold for prohibited severity.	Introduce controlled vulnerable image; pipeline should fail.
Protected GET endpoints lack auth.	High	Add authentication/RBAC dependencies.	Unauthenticated GET should return 401/403.

PHASE 15

5 Marks

Logging, Monitoring, Hardening and Secure Deployment

15.1 Security-relevant events

EVENT	PURPOSE
LOGIN / LOGIN_FAILED	Account-abuse detection.
SPOOFING_DETECTED	HMAC/device integrity failure.
LOCATION_ANOMALY	GPS/geofence/physical plausibility failure.
ROUTE_CREATED / ROUTE_MODIFIED / ROUTE_UNAUTHORIZED	Operational accountability.
DEVICE_REGISTERED / DEVICE_REVOKED	Device lifecycle.
VEHICLE_ASSIGNED / DRIVER_ASSIGNED	Dispatch traceability.
RATE_LIMIT_HIT	Availability-abuse indicator.
PRIVILEGE_CHANGE	Access-control accountability.

15.2 Metrics / alerts

METRIC	SECURITY / OPERATIONAL MEANING
API error rate	Service health.
Authentication failures	Credential or device abuse.
Telemetry rejection rate	Sensor/network/security anomaly.
Compromised / offline bins	Device health and operational risk.
429 rate-limit events	Request flooding.
CPU / memory pressure	Resource exhaustion.
Route modification frequency	Unexpected dispatch change.
Security alert count	Threat-detection workload.

15.3 Hardening checklist

Access
Protect every sensitive read/write; least privilege; review roles.

Secrets
Remove committed secrets; inject at runtime; rotate keys.

Runtime
Non-root; read-only FS; capability drop; resource limits.

Network
TLS; restrict database/broker exposure; NetworkPolicy.

IoT
Protect controller/secure element; restrict provisioning/debug access.

Monitoring
Structured security events and review process.

15.4 Physical and operational controls

- Tamper-resistant enclosure around the controller and secure element.
- Restricted access to provisioning/debug interfaces.
- Device revocation and replacement procedure.
- Controlled firmware-update path; future enhancement: signed firmware.

PHASE 16

1 Mark

Final Security Review

16.1 End-to-end traceability

REQUIREMENT	USE CASE / PROCESS	THREAT	IMPLEMENTATION	VERIFICATION
SEC-02 HMAC integrity	Submit Telemetry / P2	Device spoofing	anti_spoofing.py	test_device_signature_verification
SEC-05 Location integrity	Telemetry / Route selection	Location manipulation	validate_location_plausibility()	Location attack test
SEC-08 Object authorization	Modify Route / P3	Route tampering	RBAC + object authorization target	403 negative test
SEC-09 Rate limiting	API ingress	DoS	rate_limiter.py	429 test

REQUIREMENT	USE CASE / PROCESS	THREAT	IMPLEMENTATION	VERIFICATION
SEC-11 Audit	Route/device actions	Repudiation	AuditLog + log_audit()	Audit event test

16.2 Three highest-risk issues

RISK	CONTROL
Unauthorized route modification	Server-side authentication + role/object authorization + audit.
Device key compromise	Secure provisioning, revocation, target secure element and removal of committed secrets.
False telemetry/location reaching route logic	HMAC + freshness + geofence + plausibility + anomaly handling.

16.3 Remaining limitations / future improvements

- Current repository is simulator-first; the physical ESP32/sensors/secure-element stack is proposed rather than currently integrated.
- GPS spoofing cannot be completely eliminated by software-only checks; multi-source location and stronger device attestation are future work.
- Multi-replica deployments require shared replay state and stronger runtime secret management.
- Live Jira screenshots, burndown and final CI green evidence should be captured from the actual environment before submission.

APPENDICES

Appendix A - Project File / Phase Mapping

PROJECT FILE	PRIMARY PHASE(S)	EVIDENCE
docs/agile_process_and_development_approach.md	1, 9, 10	Agile approach, Manifesto mapping, refactoring evidence, backlog/Jira schema.
backend/app/main.py	2, 3, 5, 7, 12, 15	FastAPI endpoints, telemetry ingestion, route generation, assignment, audit, middleware.
backend/app/anti_spoofing.py	4, 7, 8, 12, 14, 15	HMAC-SHA256, replay, geofence and plausibility checks.
backend/app/auth.py	2, 5, 7, 12	bcrypt, JWT, RBAC, audit helper.
backend/app/models.py	2, 4, 5, 7	Persistent entities and relationships.
backend/app/schemas.py	2, 4, 7, 14	Typed request validation.
backend/app/route_optimizer.py	5, 8, 9, 12	Fill thresholding + geodesic Nearest-Neighbor routing.
backend/app/rate_limiter.py	7, 15	Sliding-window IP rate limiter.
backend/tests/test_unit.py	14	Unit security tests.
backend/tests/test_integration.py	14	Telemetry integration tests.
backend/tests/fuzz_test.py	14	Malformed/extreme telemetry tests.
simulator/iot_device_simulator.py	5, 7, 14	Valid, spoofed and location-manipulated telemetry.
docker/Dockerfile + docker/docker-compose.yml	11, 13, 15	Container security and runtime.

PROJECT FILE	PRIMARY PHASE(S)	EVIDENCE
k8s/deployment.yaml	13, 15	Deployment, Service, Secret, NetworkPolicy, securityContext.
.github/workflows/ci-cd.yml	11, 14	SAST, pytest, fuzz, Docker and Trivy.
frontend/index.html + app.js + styles.css	6	Dashboard, route UI, RBAC profiles, security gateway.
docs/jira_agile_board_import.json + docs/ jira_implementation_summary.md	9, 10	Epics, stories, sprints, points and goals.

Appendix B - Consistency / Reconciliation Items

ITEM	CURRENT EVIDENCE	ACTION BEFORE FINAL SUBMISSION
Backend framework	FastAPI/Uvicorn in main.py.	Use FastAPI consistently in all documentation.
Route depot	Jira acceptance target uses [13.0827,80.2707]; current route_optimizer default is (13.0,80.2).	Reconcile source/Jira/demo so one depot value is authoritative.
Role model	Frontend has analyst profile; backend UserRole enum has admin/operator/driver/viewer.	Create a server-side security-analyst role or explicitly map analyst to viewer with restricted permissions.
Protected reads	GET /api/bins, /api/routes and /api/security/logs are not protected in main.py.	Add authentication/authorization.
Device keys	SmartBin stores device_key; simulator includes a literal device key.	Move to secure provisioning and secret management; physical target can use ATECC608A.
Replay state	In-memory nonce cache.	Use shared state for multi-replica deployment.
Kubernetes secrets	Static base64 values in manifest.	Inject at deployment time.
Trivy gate	exit-code 0.	Make policy violations release-blocking.

Appendix C - Final Submission Checklist

DELIVERABLE	REPORT TREATMENT
Professional SRS	Part I - standalone requirements baseline.
Phase 1-16	Part II - every required sub-part summarized.
Diagrams	Included only in the phases where the exam requires them.
Hardware / IoT	Proposed laboratory device stack clearly separated from current simulator implementation.
Jira / Scrum	Based on uploaded project artifacts; live screenshots still need capture.
Testing	Source-backed tests documented; local environment limitation disclosed.
Security gaps	Documented explicitly as remediation targets.

References

1. 24CYS401 Secure Software Engineering End Semester Laboratory Examination - supplied instructions.
2. Secure Digital Voting System Capstone Report - supplied reference for professional academic structure.
3. Uploaded SSE-END project repository - implementation, tests, deployment and Jira artifacts.
4. OWASP Top 10 / OWASP API Security Top 10.

5. NIST SP 800-218 - Secure Software Development Framework.
6. NIST SP 800-207 - Zero Trust Architecture.

> **Evidence integrity:** the report separates implemented repository behaviour, proposed physical hardware, and recommended remediation. It does not present unsupported live metrics or unexecuted tests as completed evidence.

CONCLUSION

Engineering Conclusion

SmartWaste Secure is a secure IoT software-engineering system whose main security boundary is the transition from device-originated telemetry to trusted operational routing. The uploaded implementation already demonstrates meaningful controls: HMAC-SHA256 verification, replay checking, geofence/physical plausibility checks, JWT/bcrypt authentication, role checks, rate limiting, audit logging, container hardening and Kubernetes security controls.

The final engineering work should focus on closing the documented gaps: protect sensitive read endpoints, remove committed/static secrets, sign device freshness data, use shared replay state for replicas, reconcile the route-depot and role definitions, enforce object-level route authorization, and make security findings release-blocking. Once those changes are implemented and the corresponding Jira/CI/test evidence is captured, the phase chain will be ready for final academic submission.